

```

bubbleSort1 = [9.491E-4, 5.456E-4, 0.0132122, 0.3319656, 61.2527651];
mergeSort1 = [7.96E-5, 2.792E-4, 0.0022101, 0.0051297, 0.0624832];
selectionSort1 = [1.62E-5, 3.14E-5, 0.0029096, 0.6690401, 50.4041695];
insertionSort1 = [2.8E-6, 2.55E-5, 0.0032072, 1.0406621, 70.4065892];

bubbleSort2 = [2.614E-4, 3.929E-4, 0.0137602, 0.3550184, 36.7164078];
mergeSort2 = [2.65E-5, 1.184E-4, 0.0010892, 0.0045323, 0.048663];
selectionSort2 = [1.31E-5, 3.15E-5, 0.0029961, 0.4742685, 36.7777918];
insertionSort2 = [2.16E-5, 1.87E-5, 0.0028993, 0.4816413, 29.964836];

bubbleSort3 = [3.36E-4, 4.039E-4, 0.0130481, 0.3421517, 35.9243405];
mergeSort3 = [2.53E-5, 1.175E-4, 0.0010724, 0.0038945, 0.0438512];
selectionSort3 = [1.3E-5, 2.78E-5, 0.0025285, 0.4730089, 36.5140809];
insertionSort3 = [1.55E-5, 2.35E-5, 0.0021626, 0.4963048, 29.8540456];

bubbleSort4 = [2.416E-4, 3.819E-4, 0.0133459, 0.3368831, 44.9383076];
mergeSort4 = [3.4E-5, 1.11E-4, 0.0011236, 0.0054864, 0.0446827];
selectionSort4 = [3.63E-5, 5.27E-5, 0.002967, 0.4508136, 48.8112113];
insertionSort4 = [4.77E-5, 8.39E-5, 0.0060365, 1.0035241, 53.4213456];

bubbleSort5 = [2.643E-4, 3.832E-4, 0.0133001, 0.3562261, 35.8204588];
mergeSort5 = [2.76E-5, 1.174E-4, 0.0028262, 0.0040893, 0.0434742];
selectionSort5 = [5.7E-6, 2.81E-5, 0.0024684, 0.3727535, 43.370076];
insertionSort5 = [4.56E-5, 8.63E-5, 0.0089628, 0.9620366, 34.2878408];

bubbleSort = [bubbleSort1; bubbleSort2; bubbleSort3; bubbleSort4; bubbleSort5];
mergeSort = [mergeSort1; mergeSort2; mergeSort3; mergeSort4; mergeSort5];
selectionSort = [selectionSort1; selectionSort2; selectionSort3; selectionSort4;
selectionSort5];
insertionSort = [insertionSort1; insertionSort2; insertionSort3; insertionSort4;
insertionSort5];

% Calculate the average across the rows for each column index
bubbleSort = mean(bubbleSort, 1);
mergeSort = mean(mergeSort, 1);
selectionSort = mean(selectionSort, 1);
insertionSort = mean(insertionSort, 1);

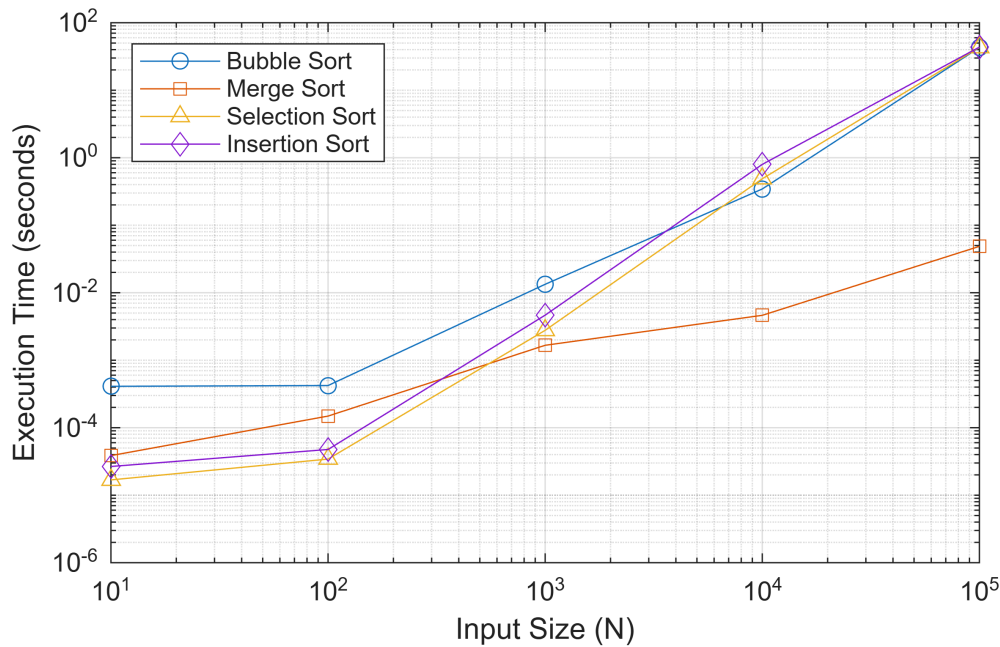
size = [10, 100, 1000, 10000, 100000];

figure();
loglog(size, bubbleSort, '-o', ...
    size, mergeSort, '-s', ...
    size, selectionSort, '-^', ...
    size, insertionSort, '-d');

xlabel("Input Size (N)");
ylabel("Execution Time (seconds)");
legend("Bubble Sort", "Merge Sort", "Selection Sort", "Insertion Sort", "Location",
"Northwest");

```

grid on;



Write-up:

With the exception of merge sort, all of the sorting algorithms have a time complexity of $O(n^2)$. Merge sort has the shortest time complexity with just $O(n \log(n))$. The run times for each sorting algorithm were tested by measuring the run time of each algorithm given a different input size (list size). There are five list sizes, starting at 10, with the size increasing by a factor of 10. The lists were generated to be filled with random numeric values, and each algorithm was given the same lists to ensure fairness. In order to gain a good perspective on how the run times changed with input size, the algorithms tested the five sizes five times each. The run times for each size for each algorithm were averaged and then plotted. As seen from the graph depicting the results, for smaller input sizes, ($n < 10^3$), there isn't a significant difference in run time. However, once input sizes started to grow, merge sort grew at a much smaller rate, an effect of its smaller run time compared to the other sorting algorithms.

Practice Problems:

1. $T(n) = \Theta(n^2)$
2. $T(n) = \Theta(n^{\log_2(7)})$
3. $T(n) = \Theta(n^2 \log_2(n))$
4. $T(n) = \Theta(n \log_2(n))$
5. $T(n) = \Theta(n^2)$
6. N/A
7. $T(n) = \Theta(n^2 (\log_2(n))^2)$

8. $T(n) = \Theta(n^{\log_2(5)})$
9. N/A
10. $T(n) = \Theta(\sqrt{n})$
11. $T(n) = \Theta((\log_2(n))^2)$
12. N/A
13. $T(n) = \Theta(\sqrt{n})$
14. $T(n) = \Theta(n(\log_2(n))^2)$
15. N/A
16. $T(n) = \Theta(\log_2(n) \sqrt{n})$
17. $T(n) = \Theta(n^{0.51})$
18. $T(n) = \Theta(n!)$
19. $T(n) = \Theta(n^{\log_2(3)})$
20. $T(n) = \Theta(n^2)$
21. $T(n) = \Theta(n \log_2(n))$
22. $T(n) = \Theta(n^2)$
23. $T(n) = \Theta(n^2)$
24. $T(n) = \Theta(2^n)$
25. $T(n) = \Theta(n^2)$